

Example 1

A portion of the JTAG debugger PEEDI's iMX6 configuration file where processor-related parameters and board components are initialised.

```
[DEBUGGER]
PROTOCOL = gdb_remote; gdb remote
REMOTE_PORT = 2000; TCP/IP port

[TARGET]
PLATFORM = CortexA8; platform is CortexA8
```

```
[PLATFORM_CortexA8]
JTAG_CHAIN = 4, 5, 4; list of TAP controllers in the JTAG chain
JTAG_CLOCK = 10000; JTAG Clock in [kHz]
TRST_TYPE = PUSH_PULL; type of TRST output: OPENDRAIN or PUSH_PULL
RESET_TIME = 100; length of RESET pulse in ms
WAKEUP_TIME = 1000; Time to delay the JTAG operations after RESET
CORE0 = iMX6A_SMP_2, 0xBA00477; TAP is Cortex-A CPU
CORE0_STARTUP_MODE = RESET; stop the core immediately after reset
CORE0_ENDIAN = LITTLE; core is little endian
CORE0_BREAKMODE = SOFT; breakpoint mode
```

```
[INIT_DDR]; initializing a DDR memory
mem write 0x020c4018 0x00260324;
mem write 0x020e05c0 0x00020000;
mem write 0x020e05b4 0x00000000;
mem write 0x020e0338 0x00000000;
mem write 0x020e0300 0x00000000;
mem write 0x020e031c 0x00000000;
mem write 0x020e0320 0x00000000;
mem write 0x020e032c 0x00000000;
mem write 0x020e05ac 0x00000000;
mem write 0x020e05c8 0x00000000
```

the kernel.

Once we have built the Linux kernel with the debug symbol, we need to have the file system also present to do source-level debugging. Once these are ready, we need to connect the target to the host system through the JTAG debugger as shown in Fig. 4.

Below are the GDB commands used in Insight console of PEEDI JTAG debugger to connect to the target, load vmlinux and start debugging.

```
(gdb) target remote <PEEDI's IP>
(gdb) cd <path of the vmlinux>
(gdb) symbol-file vmlinux
(gdb) hbreak start_kernel
(gdb) c
```

This will allow the board to boot and hit the breakpoint at start_kernel function, and from there we can start debugging. Users can set their very first breakpoint in any function of their choice, and not necessarily at start_kernel.

When debugging applications that are running in user mode, the

developer needs to start and stop all the threads related to that application at once to view the variables and the stack. It is also necessary to watch peripheral registers across different processes and the CPU. Traditional agent based debugging like GDB operates at the thread-level and can only stop a single thread. GDB cannot stop the entire system or multiple threads simultaneously.

Using JTAG we can connect to the system without changing the state of the processor registers and synchronise contexts for the Linux kernel and applications even when it is running. This helps the developer to connect JTAG when there is an error and examine the Linux kernel objects, application contexts, system calls and parameters used for system calls.

Using JTAG debuggers, we can flash the boot-loader, the Linux kernel and the Linux file system in Flash, and also debug the code present on the Flash memory by placing hard-

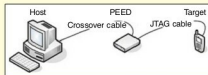


Fig. 4: Hardware set up of a JTAG Debugger

ware breakpoints and single-stepping through the code.

JTAG is here to make debugging more effective

As embedded systems become feature-dense, debugging them is also becoming increasingly difficult. Traditional agent based Linux debugging is less effective and not efficient in today's complex environment. However, using JTAG debuggers we can easily overcome these issues. We can even simplify things further by integrating a JTAG debugger with IDEs like Eclipse so we can edit, compile and debug easily in a graphic mode. IDEs help to debug the code in source level or in disassembly.

With time-to-market becoming increasingly important, companies are able to shorten the development time drastically using JTAG debuggers.

There are many players in the JTAG debuggers market, including:

1. Ronetix (<http://ronetix.at>)
2. Wind River Systems (<http://www.windriver.com>), and
3. Lauterbach (<http://lauterbach.com>)

JTAG debuggers from all these vendors vary a lot in price, differ in the number of processors supported and the IDE used for debugging. Before purchasing a hardware-debugging tool, it is good to check with the vendor about the processors supported and its Linux awareness.

The PEEDI JTAG Debugger from Ronetix is a cost-effective debugger that has almost all the features compared to the other tools present in the market. It supports a wide range of processors and works with IDEs like Eclipse, Insight, MULTI, IAR IDE, eCosPro, etc. ●

The author is technical head at Uchi Embedded Solutions, Bengaluru